

Cost-Efficient Training and Checkpointing for Large Models on Preemptible Cloud VMs

Omkar Desai
Syracuse University
New York, USA
odesai@syr.edu

Janki Bhimani
Florida International University
Florida, USA
jkbhimani@fiu.edu

Shuyi Pei
Samsung Semiconductor
California, USA
shuyi.pei@samsung.com

Bryan S. Kim
Syracuse University
New York, USA
bkim01@syr.edu

Abstract

Training large models on discounted spot VMs offers significant cost savings but remains challenging due to their low availability and unilateral preemptions. To address these challenges, we present a cost-effective training and checkpointing system for large models using spot VMs. First, we predict the preemption rate for a spot instance using historical preemption data. Second, we dynamically tune checkpointing interval through a mathematical model that uses the preemption predictions and balances the overheads of checkpointing and recovery. Finally, we guarantee consistent training throughput and minimized training cost through prediction-informed hybrid resource utilization: switching to on-demand instances when spot VM availability is low while also opportunistically scaling the number of spot VMs for training when availability is high. We improve training throughput by up to 60.27% with dynamic checkpoint interval tuning when compared to fixed-interval approaches. We also achieve up to 2.04 $\times$ higher throughput at a 51.41% lower cost through effective use of spot and on-demand instances.

CCS Concepts: • **Computing methodologies** $\rightarrow$ **Machine learning**; • **Computer systems organization** $\rightarrow$ *Dependable and fault-tolerant systems and networks*.

Keywords: Machine Learning, Cloud computing

ACM Reference Format:

Omkar Desai, Shuyi Pei, Janki Bhimani, and Bryan S. Kim. 2026. Cost-Efficient Training and Checkpointing for Large Models on Preemptible Cloud VMs. In *The 6th Workshop on Machine Learning and Systems (EuroMLSys '26)*, April 27–30, 2026, Edinburgh, Scotland Uk. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3805621.3807617>



This work is licensed under a Creative Commons Attribution 4.0 International License.

EuroMLSys '26, Edinburgh, Scotland Uk

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2605-7/26/04

<https://doi.org/10.1145/3805621.3807617>

1 Introduction

Training models with billions of parameters often demands hundreds of GPUs across several nodes, incurring high infrastructure cost [20, 24, 38–40]. Cost-reduction strategies become increasingly important as models continue to scale in size, driving up training costs [25, 53]. These strategies are especially important for democratizing ML model training and making it accessible to research labs and small to medium sized organizations with limited budgets. One practical approach in cloud environments is leveraging discounted spot VMs; these instances offer unused cloud resources at a discounted rate (up to 90% compared to on-demand price), making them particularly compelling for cost-sensitive deployments [3, 7, 35].

Using spot VMs for large-scale training is non-trivial due to their low availability and frequent preemptions. These VMs only exist when providers have excess resources and can be preempted any time their demand increases, causing training to halt [41, 43, 50]. This makes predicting training time when training on spot VMs impossible. Additionally, the high preemption probabilities for GPU instances (exceeding 20% over a 30-day window) and the short preemption notices (0-3 minutes) provided by service providers make it challenging to reliably persist model state right before an instance is preempted [5]. While periodic checkpointing of model states provides a natural recovery mechanism for spot VM preemptions, Figure 1 shows that checkpointing has become increasingly resource-intensive as checkpoint sizes grow with model scale, placing heavy demands on memory, storage, and I/O [26, 27, 31, 47]. Consequently, minimizing checkpointing overheads has become a critical requirement.

Prior works mitigate some of the challenges faced by training on spot VMs. Works on checkpointing optimize I/O [31, 47], tune frequency via user-defined parameters [32, 42], or leverage redundancy in data-parallel training for partial recovery [8, 19, 48], but do not consider preemption rates to

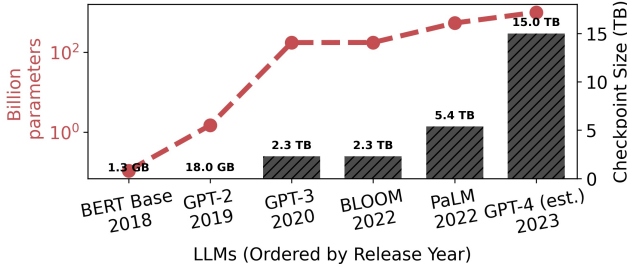


Figure 1. Popular models and their checkpoint sizes over the past 7 years. Models have exploded in size and consequently their checkpoints too, making checkpointing challenging [1, 9, 10, 14, 37, 49].

tune the checkpointing interval (CI). Similarly, other systems manage spot volatility through parallelism reconfiguration [2, 16, 21, 45] or performing redundant computations [44], but still cause training to halt when spot VMs become unavailable.

Consequently, existing works that leverage spot instances for training are unable to effectively (1) minimize checkpointing overhead and preemption recovery costs, and (2) provide training time predictability despite low spot availability while also opportunistically accelerating training when spot instances are abundant. The lack of training time predictability and a high preemption recovery overhead limits the use of spot instances for training models.

Our work aims to **enable cost-effective large model training on spot VMs without sacrificing training time predictability**. Our key insight is that preemption rates can be predicted and leveraged to jointly optimize checkpointing intervals and resource provisioning decisions. By using predicted preemption rates, we dynamically adjust CI to minimize frequency during stable periods and increase frequency before predicted disruptions. We also design a resource manager that uses the predictions and maximizes spot VM use to cost-effectively accelerate training when preemptions are rare and falls back to on-demand VMs to ensure training continues even when spot instances are unavailable.

We make the following contributions:

- We develop a lightweight time-series forecasting approach that predicts spot instance preemptions by analyzing historical preemption patterns, enabling adaptation of checkpointing and provisioning strategies.
- We formulate a mathematical model used to dynamically determine CI in distributed training based on predicted preemption rates, explicitly balancing the trade-off between checkpointing overhead and expected recovery costs.
- We design a novel resource manager that opportunistically scales data parallel (DP) shards to accelerate training when spot instances are abundant and falls

back to on-demand when availability is low to ensure training progress. Tensor parallel (TP) and pipeline parallel (PP) configuration is frozen to prevent expensive model repartitioning.

Our system is implemented on top of PyTorch and uses TorchElastic for resource management [34]. We compare against PCCheck [42], Parcae [16], and CheckFreq [32] on LLMs up to 12.8B parameters. Our evaluations show a 2.04× higher training throughput while reducing training costs by 51.41% when compared to training exclusively on on-demand. Furthermore, our system improves training throughput by up to 60.27% compared to fixed-interval checkpointing by dynamically tuning the CI. Our system will be open sourced.

2 Background and related works

This section provides a brief overview of large model training on spot VMs using elastic training. We also highlight how prior works address some of the challenges.

2.1 Preemptible spot virtual machines

Spot VMs are a cost-effective way to use cloud resources for training models as they are available at significantly reduced prices compared to their on-demand counterparts. Spot instances are commonly available at up to 70-90% discounts compared to on-demand pricing [3, 7, 35], but they are available only when providers have excess capacity. Furthermore, they can be preempted at any time by the cloud providers on a short notice (0-3 minutes) [41, 43].

Prior works mitigate some of the challenges associated with training on spot VMs, but this remains insufficient [30, 46, 51]. FastPersist [47] and DatastatesLLM [31] optimize I/O and hide checkpointing overheads using DRAM for snapshots, but risk memory saturation as model sizes grow. Other systems optimize I/O and checkpoint frequency (PCCheck [42], CheckFreq [32]), though they rely on static, user-defined overhead parameters for CI determination. Alternatively, redundancy-based approaches exploit the inherent redundancy of DP shards for partial failure recovery (AdaCheck [29], Gemini [48], CheckMate [8], Just-In-Time [19]); however, these require massive scale and are impractical for smaller deployments (e.g., 1-3 DP shards). Varuna [2], Tenplex [45], Parcae [16], and Oobleck [21] manage spot VMs via parallelism reconfiguration and Bamboo [44] uses redundant computations (1.5×) to tolerate preemptions, but in all cases, training halts when spot VMs become unavailable.

2.2 Elastic training for ML models

Elastic training dynamically scales the number of workers in a distributed training job without pausing the entire process. This is useful in spot environments, where workers can be preempted or become available as demands change. By adding workers when spot instances are abundant, training can be accelerated; when workers are preempted, training can continue on the remaining workers without full job

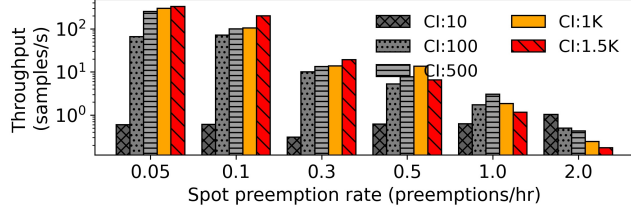


Figure 2. The optimal CI depends on the preemption rates. Other factors such as checkpointing and recovery overheads also have an impact. Y-axis shown in Log scale.

restarts. None of the prior works have used elastic training to achieve predictability in training time with spot instances.

TorchElastic [36] enables elastic training for distributed PyTorch. When the set of active workers changes, it reconfigures parallelism configuration and resumes training from the last synchronized state. We use TorchElastic exclusively on the DP dimension, scaling the number of DP shards while keeping TP and PP dimensions fixed throughout training. This enables our work to scale training based on preemption prediction without the prohibitive reconfiguration overhead associated with changing TP and PP dimensions.

3 Motivation

We now demonstrate the challenges in large model checkpointing on spot instances using an Azure NC series VM [6] with 4×A100 GPUs [33] and spot VM availability traces from SkyPilot [50].

3.1 Need for dynamically tuned checkpoint interval

Spot VM preemption rates vary and the optimal checkpointing interval depends on the preemption rate. Therefore, when training using spot VMs, checkpointing interval must not be a static user-defined parameter, as it commonly is [42]. Figure 2 shows training throughput for training a 1B parameter GPT-3 model [9] on the Azure VM at different checkpointing intervals as spot preemption rates vary. We use PyTorch and train with $DP \times PP \times TP = 4 \times 1 \times 1$. We observe that the ideal checkpointing interval depends on the preemption rate and a suboptimal interval can slow down training by up to 2.74 orders of magnitude (preemption rate of 0.05 preemptions/hr and CI:10). These results highlight the importance of dynamically adapting the checkpoint interval in response to varying preemption rates to balance checkpointing and recovery overheads and maximize training throughput.

3.2 Need for adaptive spot tuning

While attractive for saving costs, the inherent nature of spot VMs makes it challenging to train models with a predictable training time. Spot VM availability fluctuates widely, making training on them unpredictable. Figure 3 shows spot availability from the SkyPilot traces [50] for a request block of 16

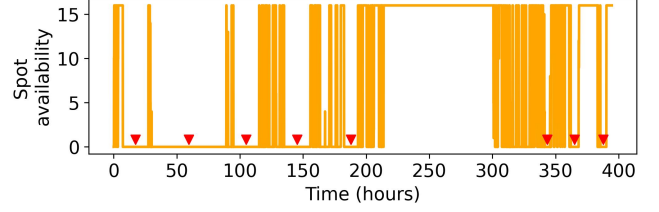


Figure 3. Spot availability for 16 p3.2xlarge GPU nodes in AWS us-west-2c region over 2 weeks. Red arrows show periods of no spot VM availability for more than 4 hours.

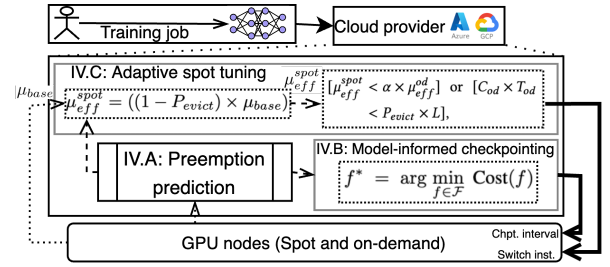


Figure 4. System architecture.

AWS p3.2xlarge [4] instances over 2 weeks with the red arrows showing periods of no availability for more than 4 hours. We observe that spot instances can go from being highly available to mostly unavailable over a short time. As a result, exploiting the low cost of spot instances becomes challenging without compromising on training time predictability [50]. This motivates the need to leverage on-demand instances during periods of spot unavailability and opportunistically maximize spot instance use during periods of high spot availability to ensure predictable training time and to accelerate training cost-effectively.

4 Design

Our work consists of three components: (1) Time-series forecasting to predict spot preemption rate, (2) mathematical model-informed checkpointing that adapts CI based on predicted preemption rate to maximize training throughput, and (3) hybrid resource management that uses spot and on-demand VMs to combine the training time predictability of on-demand VMs with the cost-benefit of spot VMs. Figure 4 illustrates the overall system architecture and Table 1 summarizes model parameters and their descriptions used in our mathematical model. Preemption rate predictions drive checkpoint interval optimization and hybrid resource provisioning decisions. Our system is implemented on PyTorch and uses TorchElastic for instance type switching and scaling.

4.1 Preemption prediction

We now describe how we use time-series forecasting to predict preemption rates. While prior work predicts aggregate spot availability for parallelism reconfiguration [16], our goal is different: we forecast preemption rate experienced by a user for an instance type to inform checkpoint frequency and scaling decisions.

Inspired by prior work, we leverage statistical time-series forecasting instead of complex neural network models for this prediction. Specifically, we use Seasonal ARIMA (SARIMA) as our predictor due to its ability to capture the daily and weekly temporal patterns observed in spot preemption data.

For predictions, training time is split into hour intervals with interval length $P_T = 1$ hour. Since raw hourly preemption counts are sparse, we apply a W -hour causal rolling mean to obtain a continuous rate r_i before fitting. Preemption rate logging over the past H intervals is captured as $(r_{i-H}, \dots, r_{i-1})$ and used to predict the preemption rate λ_t for the next interval as $\lambda_t = \text{PREDICTION}(r_{i-H}, \dots, r_{i-1})$.

Preemption event logging specifically records spot VM availability count from a maximum request size by sampling every g seconds. We then derive the raw preemption count for sample i as $c_i = \max(0, a_{i-1} - a_i)$ where a_i is the availability at sample i . Raw counts are aggregated into hourly intervals to obtain $r_i = \sum c_j$ over all samples j within hour i . The raw hourly preemption series is highly sparse (80% of hours have zero preemptions), which causes SARIMA to produce near-zero predictions. A W -hour causal rolling mean before fitting converts this sparse spike series into a continuous rate signal and is sufficient to predict a coarse grained preemption rate. For this work, we use $W=3$ hours and get a smoothed preemption rate over time $\tilde{r}_i$. We fit $\text{SARIMA}(1, 1, 1)(1, 0, 1)_{24}$ on $\tilde{r}_i$. The seasonal period $s = 24$ captures daily preemption periodicity, and the $d = 1$ differencing term handles drift in the smoothed rate over time, which is not present in the raw sparse series.

We continuously log preemption events during runtime and retrain the forecasting model weekly using a sliding window of the most recent $M = 4$ weeks of data to adapt to evolving preemption patterns. This periodic retraining is lightweight and takes less than one minute on a single CPU core.

4.2 Model-informed checkpointing

Next, we present a high-level mathematical model of asynchronous checkpointing for distributed ML model training that dynamically tunes CI based on the predicted preemption rate. The model determines an optimal CI based on (1) preemption likelihood, (2) hardware performance parameters of the training nodes (including CPU, GPU, memory, and storage), and (3) training characteristics (iteration time, computational cost). The key idea is balancing two competing costs: when preemption rates are high, frequent

Table 1. Summary of notations.

Parameter	Unit	Description
λ_t	Premp./hr	Spot preemption rate
S	GB	Size of one checkpoint
C	GB	Total DRAM capacity allocated for checkpointing
T	Sec	Time for one training iteration
D	Sec	Time to copy checkpoint from GPU to DRAM
S_t	Sec	Time to copy checkpoint from DRAM to storage
f	Iter./chpt.	Checkpointing Interval
N_{iter}	-	Total number of training iterations
μ_{base}	Samples/sec	Baseline training throughput
N_{dp}	-	Number of DP shards in training job

checkpointing reduces recovery overhead; when preemption rates are low, infrequent checkpointing minimizes overhead from checkpoint writes. Unlike the classical Young-Daly formula [11, 15, 52], which assumes a static failure rate and synchronous checkpointing, our model accounts for the DRAM buffering constraints presented by asynchronous checkpointing and adapts to time-varying preemption rates. Our model is empirically validated against real hardware.

4.2.1 Training time. Training time is estimated based on the average iteration time per DP shard (T/N_{dp}). This estimate considers long term training with failures and checkpointing overheads and no network bottlenecks. We start with λ_t from §4.1 and determine preemption rate per iteration as $\lambda_{iter} = \lambda_t \times (T/3600)$. To recover from an interruption at checkpointing interval f , we expect to rerun $f/2$ iterations on average, each costing T/N_{dp} seconds. Consequently, the amortized recovery time per iteration due to interruptions is given by Equation 1:

$$\lambda_{iter} \times \left(\frac{f}{2} T/N_{dp} \right) = \frac{\lambda_{iter} \times f(T/N_{dp})}{2} \quad (1)$$

Next, we incur a cost of D seconds to copy a checkpoint from GPU to DRAM. When checkpointing every f iterations, that per iteration cost can be given by D/f .

Finally, the per iteration time must also include the checkpointing time that can not be hidden behind the training time. When the DRAM allocated for checkpoint buffering is full and training time between checkpoints is less than time to copy a checkpoint to storage ($f \times (T/N_{dp}) < S_t$), we have to wait until the previous checkpoint is persisted and frees up S GB in DRAM. For each iteration, this is given by Equation 2:

$$T_{stall} = \begin{cases} 0 & \text{if } f \times (T/N_{dp}) \geq S_t \\ \frac{S_t}{f} - (T/N_{dp}) & \text{otherwise} \end{cases} \quad (2)$$

Putting it all together, the expected training time when checkpointing every f iterations is given by Equation 3:

$$\text{Cost}(f) = (T/N_{dp}) + \frac{D}{f} + T_{stall} + \frac{\lambda_{iter} \times f(T/N_{dp})}{2} \quad (3)$$

The effective throughput in samples/second is then simply given by $\text{Throughput}(f) = 1/\text{Cost}(f)$.

4.2.2 Memory constraints. The number of checkpoints that can be buffered into DRAM in asynchronous checkpointing depends on the memory available for checkpointing and must be considered to determine the interval. Each checkpoint is S GB, and the buffer may hold up to $S_t/(f(T/N_{dp}))$ checkpoints. Formally, we require $S_t/(f(T/N_{dp})) \leq C$, which gives a lower bound on the checkpointing interval: $f \geq \frac{S_t S}{C(T/N_{dp})}$. Violating this lower bound results in training stalls (Equation 2). Since f must be a positive integer, it is restricted to the set $\mathbb{Z}^+$, and is given by Equation 4:

$$\mathcal{F} = \left\{ f \in \mathbb{Z}^+ : f \geq \lceil (S_t S)/(C(T/N_{dp})) \rceil \right\} \quad (4)$$

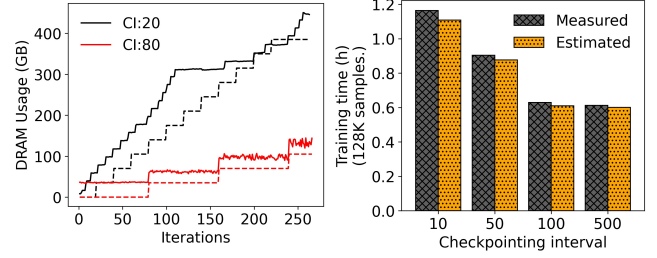
The optimal interval f^* at any given time during training is found by sweeping over $\mathcal{F}$ and is given by Equation 5. We use this model within a closed-loop and adapt checkpointing interval whenever preemption prediction changes during runtime.

$$f^* = \arg \min_{f \in \mathcal{F}} \text{Cost}(f) \quad (5)$$

4.2.3 Model validation. For the model validation, we use a CloudLab d8545 [17] system with 4×A100 40GB GPUs [33], 2×AMD EPYC 7413 CPUs [13], 512 GB DRAM, and remote storage for checkpoints. We fix checkpointing to occur every 20 and 80 iterations for this validation.

Figure 5a shows that our model accurately tracks DRAM usage during checkpointing. The small gap between measured (solid) and modeled (dotted) memory usage comes from profiling differences: we measure full system DRAM, while our model only captures checkpoint buffer size. Capturing framework-level allocation would further reduce this difference. Overall, the model estimated DRAM utilization closely aligns with real behavior, with correlation coefficients of 0.955 and 0.990.

Figure 5b demonstrates that our model also estimates end-to-end training time under different checkpointing configurations with high accuracy. We measure training time without failures at different checkpointing intervals for training over 128K samples. Model estimated time for 128K samples stays within 5.45% of measured training time, and as expected, longer checkpoint intervals reduce checkpoint overhead and shorten training time. Together, these results demonstrate that our mathematical model accurately estimates the performance impact of checkpointing in real systems.



(a) Estimated DRAM use (dotted lines) against real measurements (solid lines). (b) Estimated training time against measured while varying checkpoint intervals.

Figure 5. Model validation for different checkpoint intervals.

4.3 Adaptive spot tuning

Finally, we describe our resource manager which ensures predictability in training time despite low spot availability while also cost-effectively accelerating training when spot availability is high. Our approach adapts to changing spot preemption rates and utilizes a mix of spot and on-demand instances to (1) achieve on-demand like predictability in training time while using spot instances, and (2) opportunistically boost training performance by leveraging spot instances when abundantly available. To avoid expensive parallelism reconfiguration, we treat each TP×PP GPU group as the atomic scaling unit and restrict elasticity to DP shard multiples. Fine-grained re-sharding across PP and TP dimensions is left for future work.

Low spot instance availability causes training jobs to stall and using on-demand instances is critical to ensure training progress when spot instances are unavailable. Therefore, when predicted preemption rate (λ_t) is high and can cause training stalls or throughput to fall significantly below a baseline (μ_{base}), we switch DP shards with at-risk spot VMs to on-demand instances to ensure training continues. μ_{base} determines the minimum number of DP shards and nodes ($nodes_{low}$) needed to achieve the baseline throughput. Our objective is to maintain a user defined baseline throughput (μ_{base}) while minimizing cost.

For spot instances, we assume the same throughput ($\mu_{base} = 1/T$) as on-demand instances as the hardware for both is identical. In case of spot instances, preemptions reduce μ_{base} by losing progress and incurring recovery overheads. Therefore, the effective spot instance throughput is discounted by the risk of preemption, as given by Equation 6.

$$\mu_{eff}^{spot} = ((1 - P_{evict}) \times \mu_{base}) \quad (6)$$

Here spot preemption probability (P_{evict}) is given by $P_{evict} = 1 - e^{-\lambda_t}$ assuming preemptions follow a Poisson process given the independent nature of spot preemptions [22].

On-demand instances on the other hand sustain the full throughput $\mu_{eff}^{od} = \mu_{base}$ as they are not preemptible. Our

objective is to maximize spot use while also ensuring a minimum progress rate μ_{base} . At each decision point, we compare the throughput of training on spot and the expected cost-benefit of switching. We switch a DP shard to on-demand instances only if:

$$[\mu_{eff}^{spot} < \alpha \times \mu_{eff}^{od}] \text{ or } [C_{od} \times T_{od} < P_{evict} \times L] \quad (7)$$

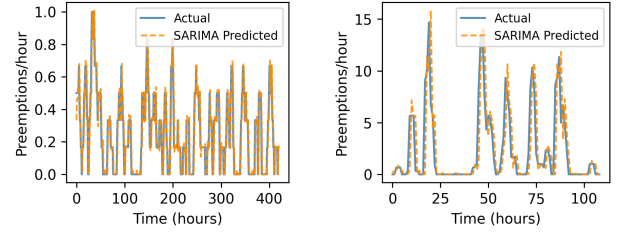
Here α is the throughput degradation threshold to avoid the frequent and unnecessary switching costs which differ across cloud providers based on implementation details (we swept α from 0.6–0.95 and found results stable in the 0.7–0.9 range; we report 0.8 as the midpoint). C_{od} and C_{sp} are the on-demand and spot cost rate respectively, T_{od} is the fixed duration of on-demand use before attempting to switch back to spot, and L is the monetary loss incurred when a spot instance fails. This is given by the sum of the compute lost during the failure and the compute required for recovery. Formally, $L = (T_{wasted} \times C_{sp}) + (T_{recover} \times C_{od})$ where T_{wasted} is the time elapsed since the last successful checkpoint, and $T_{recover}$ is the expected time to recover from the previous checkpoint. The maximum number of on-demand nodes never exceeds $nodes_{low}$.

When spot instances are abundant, we opportunistically accelerate training by scaling the number of DP shards to increase throughput over μ_{base} . The key idea is that spot VMs can be used only when they are available and therefore we should leverage them whenever abundant to speed up training cheaply. We greedily scale DP shards up to an upper bound ($nodes_{high}$) which is a multiple of $PP \times TP$ using spot VMs whenever preemption probability is below 10%.

5 Evaluation

For our preliminary evaluations, we use the Microsoft Azure NC96ads_A100_v4 VMs [6] with 4×80GB A100 GPUs [33], 96 vCPUs (AMD EPYC 7V13) [13], 880 GB DRAM, and remote SSD backed NFS for dataset and checkpoint storage. We compare our work against (1) Parcae: exclusively using spot VMs and reconfiguring PP configuration in response to preemptions, (2) PCCheck: using concurrent I/Os to enable frequent checkpointing, and (3) CheckFreq: asynchronous checkpointing and interval determination to keep overhead under a user specified limit. We answer the following questions:

- Is the preemption rate predictor accurate? (5.1)
- How does checkpoint interval adapt to dynamicity in spot preemption rates, and how does it impact throughput (5.2)
- Does our system switch between spot and on-demand VMs to maximize throughput and minimize training cost? (5.3)



(a) 1 node 45 days.

(b) 16 nodes, 11 days.

Figure 6. SARIMA-predicted vs. actual preemption rate.

5.1 Accuracy of preemption rate prediction

We train and evaluate our predictor on long and short SkyPilot traces, achieving mean absolute error (MAE) of 0.13 (420 hours, 1 node) and 1.32 (271 hours, 16 nodes) for 1-hour-ahead predictions. Figure 6 shows that SARIMA’s predictions closely track actual preemption rates, and are accurate even with limited historical data. This prediction is used by our model for determining CI, and by our resource manager for instance switching and scaling decisions. Furthermore, our work is robust to prediction errors as the predicted preemption rate is only one component in the overall decision.

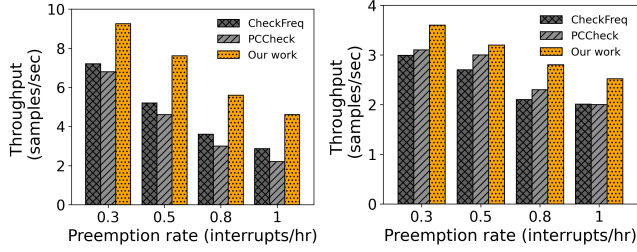
5.2 Benefits of dynamic checkpointing

We evaluate dynamic checkpointing interval optimization through two experiments. First, we evaluate interval selection across model sizes and preemption rates. Second, we demonstrate the continuous adaptation of the checkpointing interval to time-varying preemption patterns.

We train two GPT-3 models (6.65B and 12.84B parameters) with parallelism configurations $DP \times TP = 2 \times 2$, and $TP = 4$, respectively. We train each model for four hours with results averaged over three runs. We inject preemptions at controlled rates ($\lambda_t \in \{0.3, 0.5, 0.8, 1.0\}$ preemptions/hour) and use a simple process monitor that restarts training after failures. We compare our work against PCcheck and CheckFreq, both configured with 5% maximum overhead.

Figure 7 shows that our work consistently delivers the highest throughput across all model sizes and preemption rates. For the 6.65B model (7a), we achieve 60.27% higher throughput at $\lambda_t = 1.0$ as adaptive intervals mitigate the high recovery penalty. For the 12.84B model (7b), gains are smaller (25.37% at $\lambda_t = 1.0$) because async checkpointing overhead is hidden by higher compute cost of the larger model. Table 2 shows checkpoint intervals chosen by each approach - CheckFreq and PCcheck have static intervals, while our system dynamically adapts the interval to preemption rates.

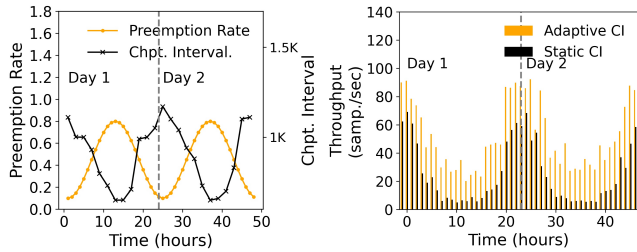
We next train a 1B parameter model ($DP=4$) under a synthetic preemption pattern simulating diurnal preemptions



(a) GPT3 - 6.65 billion (4 hours) (b) GPT3 - 12.84 billion (4 hours)

Figure 7. Training throughput for GPT-3 LLMs on an Azure Standard_NC96ads_A100_v4 VM for different failure rates (not including failure recovery) averaged over three runs.**Table 2.** Checkpointing intervals selected by different techniques under varying failure rates for GPT-3 6.65 B model.

Checkpoint interval determination technique	Failure rate			
	0.3	0.5	0.8	1
Our work	760	710	670	419
CheckFreq	677 for all failure rates			
PCCheck	700 for all failure rates			



(a) Checkpointing interval adapt (b) Training throughput over changing preemption rate

Figure 8. Impact of adaptively tuning checkpointing interval in response to changing VM preemption rates.

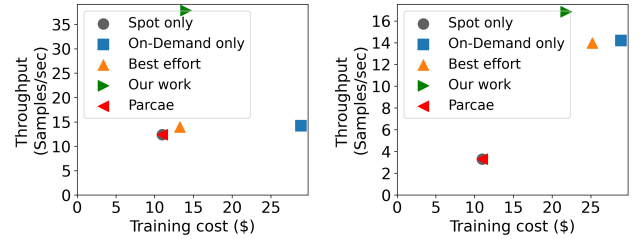
over two days (from $\lambda_t = 0.1$ to $\lambda_t = 0.8$) and show the results in Figure 8. Figure 8a shows the model-determined CI tracking preemption rates: intervals lengthen during stable periods and shorten during high-risk periods. This adaptation yields 2.04× average throughput improvement over a fixed interval baseline as shown in Figure 8b.

5.3 Benefits of hybrid resource management

We evaluate our hybrid resource manager by training a 6.65B parameter GPT-3 model (TP×DP=2×2) for 100K samples on Azure nodes with 4×A100 GPUs. We scale from 1-8 nodes (4-32 GPUs), adding DP shards and use spot preemption traces from SkyPilot [50] — one with high availability (74.9%) and one with low (38.3%). Table 3 summarizes the

Table 3. SkyPilot multi-node spot VM trace characteristics. Highlighted rows represent traces used for evaluation.

Trace name	Availability	Interrupts/hr	Avg. available duration	Avg. capacity
us-east-2b_v100_1	62.8	0.18	3.47	8.76
us-west-2a_v100_1	44.6	0.25	1.78	6.90
us-west-2c_v100_1	44.0	0.23	1.93	6.83
us-west-2c_v100_1	83.9	0.18	4.60	3.26
us-east-1f_v100_1	38.3	0.14	2.66	1.38
us-east-2a_v100_1	74.9	0.03	21.36	2.92



(a) High spot availability and max node limit 8. (b) Low spot availability and max node limit 4.

Figure 9. Multi-node training throughput vs cost for a 6.65B model using different preemption traces.

characteristics of the analyzed traces with highlighted rows indicating the ones selected for evaluation. The us-east-1f trace (38.3% availability, 0.14 interrupts/hr, 1.38 average capacity) represents a low-availability scenario where spot instances are scarce and frequently preempted, forcing our system to rely heavily on on-demand fallback to maintain training progress. The us-east-1a trace (74.9% availability, 0.03 interrupts/hr, 2.92 average capacity) represents a favorable high-availability scenario where spot instances are abundant and stable, allowing the resource manager to opportunistically scale training.

We compare against four approaches: (1) Parcae, (2) spot-only, (3) on-demand-only, and (4) best-effort hybrid (switches to on-demand when spot unavailable but no opportunistic scaling).

Figure 9 shows that our system achieves Pareto optimality, delivering the best performance per dollar. Under high availability (Figure 9a), we improve throughput by 2.04× at 51.41% lower cost than on-demand-only and 1.71× higher than best-effort at only 5.88% higher cost. Under low availability (Figure 9b), we maintain 18.73% higher throughput at 24.56% lower cost than on-demand-only. Parcae performs similarly to spot-only as it lacks on-demand fallback mechanisms, causing training to stall when spot instances becomes unavailable. Note that Parcae is designed for multi-node PP and our single-node PP configuration limits its reconfiguration benefits. Our work and Parcae address fundamentally complementary challenges (checkpointing vs parallelism re-configuration) and can be combined together.

Table 4. Comparison of training time, cost-efficiency, run-time, and switching overhead for training with low and high spot VM availability.

Spot availability:	High			Low		
	Training time (h)	Cost-eff. (thr./\$)	Switching overhead	Training time (h)	Cost-eff. (thr./\$)	Switching overhead
On-demand only	1.98	0.49	–	1.98	0.49	–
Spot-only	2.24	0.30	–	8.42	1.12	–
Best effort	1.99	0.55	7.2%	1.98	1.05	10.6%
Parcae	2.24	0.30	–	8.42	1.12	–
Our work	0.73	0.77	12.3%	1.65	2.69	16.6%

Table 4 shows we deliver 40% better performance-per-dollar than best-effort under low availability and 140% better than spot-only under high availability. Training time is reduced by 63% under high availability and 17% under low availability. Improvements are small under low availability as fewer spot VMs are available. We also report switching overheads for approaches that scale VMs during training. Our approach incurs higher overhead (12.3–16.6%) than best-effort (7.2–10.6%) because it performs opportunistic scale-up and not just on-demand fallback. This overhead is offset by the high cost-efficiency achieved by our approach.

6 Discussions and future work

While our evaluations focus on dense transformer models, our contributions—preemption prediction, checkpoint interval tuning, and hybrid resource management—operate at the infrastructure and training framework levels, independent of which model architecture is being trained. Therefore, our work can be directly applied to new model architectures such as Mixture-of-Experts (MoE) and multimodal models.

While dense models consist of several layers in a single model, MoE models consist of multiple, generally smaller subject matter experts. Unlike training dense models where all layers are updated after every training iteration, these models activate only a subset of experts for every training iteration based on the relevance of the training data to the individual experts. Despite this, checkpointing MoE models involves persisting the entire model regardless of which experts are activated during an iteration. For a MoE model such as Mixtral 8×7B [23] with 47B total parameters, the checkpoint size S reflects the complete model with 47B parameters and not just a specific expert. Multimodal models such as LLaVA [28] on the other hand are designed for multiple modalities of data. These models jointly train multiple modality-specific components such as vision, audio, and language modals on heterogeneous data batches spanning images, audio, and text. Similar to MoE models, all modality components must be checkpointed at every CI regardless of the modality composition of the training batch.

While the fundamentals of our contributions generalize to both, MoE and multimodal models, architecture-specific optimizations can further improve performance. For MoE

models, expert utilization is skewed—frequently invoked experts contribute more to training progress and carry higher recovery cost if lost [18]. Checkpointing frequently used experts more often than their low-utilization counterparts would reduce the effective checkpoint size per interval while preserving the most critical model state. For multimodal models, subcomponents such as vision encoders and language decoders have different computational profiles and memory footprints [12], and are well suited to heterogeneous hardware. Provisioning heterogeneous spot instances—pairing GPU-heavy nodes for GPU-intensive components with CPU-optimized nodes for preprocessing—could improve cost efficiency beyond what a homogeneous spot pool allows.

Several other limitations and directions remain open. Our system scales exclusively along the DP dimension, which limits how aggressively it can leverage spot instances. Integrating TP/PP reconfiguration, as in Parcae, could enable finer-grained scaling and better spot utilization. Extending the system with SLO-aware resource management where users specify a target training completion time and the system automatically determines degree of parallelism, spot-to-on-demand ratio, and scaling strategy is a natural next step toward making spot-based training more practical and cost-efficient.

7 Conclusion

We presented a system for cost-efficient large model training on spot VMs that addresses two key challenges: (1) dynamically adapting CI to predicted preemption rates, and (2) balancing spot and on-demand instances to maintain consistent training progress and cost-effectively accelerate training when possible. Our approach achieves up to 2.04× higher throughput with 51.41% cost reduction through hybrid resource management. We demonstrate that preemption-aware checkpointing, combined with elastic scaling, can make spot-based training practical.

Acknowledgments

We would like to thank the anonymous reviewers for their constructive feedback. We also thank Syracuse University Information Technology Services (ITS) for providing computing resources and support. This research was supported in part by the National Science Foundation under award numbers CAREER-2443515, CAREER-2338457, CSR-2402328, and CSR-2323100. This work is also supported through the Global Research Support Program in the Digital Field supervised by the Institute for Information and Communications Technology Planning and Evaluation (IITP) under Grant RS-2024-00428758, funded by the Ministry of Science and ICT (MSIT), South Korea. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the supporting agencies.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Sanjith Athlur, Nitika Saran, Muthian Sivathanu, Ramachandran Ramjee, and Nipun Kwatra. Varuna: scalable, low-cost training of massive deep learning models. In *Proceedings of the Seventeenth European Conference on Computer Systems*, pages 472–487, 2022.
- [3] AWS. Amazon ec2 spot instances pricing. <https://aws.amazon.com/ec2/spot/pricing/>, 2025.
- [4] Amazon Web Services (AWS). Amazon EC2 P3 instances. accelerate machine learning and high performance computing applications with powerful GPUs. <https://aws.amazon.com/ec2/instance-types/p3/>, 2023.
- [5] Amazon Web Services (AWS). Amazon ec2 spot instances. <https://aws.amazon.com/ec2/spot/instance-advisor/>, 2025.
- [6] Azure. Nc_a100_v4 sizes series. <https://learn.microsoft.com/en-us/azure/virtual-machines/sizes/gpu-accelerated/nc100v4-series?tabs=sizebasic>, 2024.
- [7] Azure. Azure spot virtual machines pricings. <https://azure.microsoft.com/en-us/pricing/spot-advisor/>, 2025.
- [8] Ankit Bhardwaj, Weiyang Wang, Jeremy Carin, Adam Belay, and Manya Ghobadi. Checkmate: Zero-overhead model checkpointing via network gradient replication. *arXiv preprint arXiv:2507.13522*, 2025.
- [9] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [10] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *Journal of machine learning research*, 24(240):1–113, 2023.
- [11] John T Daly. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future generation computer systems*, 22(3):303–312, 2006.
- [12] Omkar Desai, Ziyang Jiao, Shuyi Pei, Janki Bhimani, and Bryan S Kim. Preparation meets opportunity: Enhancing data preprocessing for ML training with seneca. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*, pages 221–238, 2026.
- [13] Advanced Micro Devices. AMD EPYC 7413. <https://www.amd.com/en/products/processors/server/epyc/7003-series/amd-epyc-7413.html>, 2024.
- [14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- [15] Jack Dongarra, Thomas Herault, and Yves Robert. Fault tolerance techniques for high-performance computing. In *Fault-tolerance techniques for high-performance computing*, pages 3–85. Springer, 2015.
- [16] Jiangfei Duan, Ziang Song, Xupeng Miao, Xiaoli Xi, Dahua Lin, Harry Xu, Minjia Zhang, and Zhihao Jia. Parcae: Proactive, Liveput-Optimized DNN training on preemptible instances. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 1121–1139, 2024.
- [17] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, et al. The design and operation of CloudLab. In *USENIX Annual Technical Conference (ATC)*, pages 1–14, 2019. <https://www.usenix.org/conference/atc19/presentation/duplyakin>.
- [18] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- [19] Tanmaey Gupta, Sanjeev Krishnan, Rituraj Kumar, Abhishek Vijeev, Bhargav Gulavani, Nipun Kwatra, Ramachandran Ramjee, and Muthian Sivathanu. Just-in-time checkpointing: Low cost error recovery from deep learning training failures. In *Proceedings of the Nineteenth European Conference on Computer Systems*, pages 1110–1125, 2024.
- [20] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, and Zhifeng Chen. GPipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.
- [21] Insu Jang, Zhenning Yang, Zhen Zhang, Xin Jin, and Mosharaf Chowdhury. Ooblock: Resilient distributed training of large models using pipeline templates. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 382–395, 2023.
- [22] Myeongjae Jeon, Shivaram Venkataraman, Amar Phanishayee, Junjie Qian, Wencong Xiao, and Fan Yang. Analysis of Large-Scale Multi-Tenant GPU clusters for DNN training workloads. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 947–960, 2019.
- [23] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- [24] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, et al. MegaScale: Scaling large language model training to more than 10,000 GPUs. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 745–760, 2024.
- [25] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [26] Wenshuo Li, Xinghao Chen, Han Shu, Yehui Tang, and Yunhe Wang. ExCP: Extreme LLM checkpoint compression via weight-momentum joint shrinking. *arXiv preprint arXiv:2406.11257*, 2024.
- [27] Xinyu Lian, Sam Ade Jacobs, Lev Kurilenko, Masahiro Tanaka, Stas Bekman, Olatunji Ruwase, and Minjia Zhang. Universal checkpointing: Efficient and flexible checkpointing for large scale distributed training. *arXiv preprint arXiv:2406.18820*, 2024.
- [28] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023.
- [29] Weijie Liu, Shengwei Li, Zhiqian Lai, Keshi Ge, Qiaoling Chen, Peng Sun, Dongsheng Li, and Kai Lu. AdaCheck: Adaptive checkpointing for large language model training. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*, 2026.
- [30] Luo Mai, Guo Li, Marcel Wagenländer, Konstantinos Fertakis, Andrei-Octavian Brabete, and Peter Pietzuch. KungFu: Making training in distributed machine learning adaptive. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 937–954, 2020.
- [31] Avinash Maurya, Robert Underwood, M Mustafa Rafique, Franck Cappello, and Bogdan Nicolae. Datastates-LLM: Lazy asynchronous checkpointing for large language models. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*, pages 227–239, 2024.
- [32] Jayashree Mohan, Amar Phanishayee, and Vijay Chidambaram. Check-Freq: Frequent, Fine-Grained DNN checkpointing. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 203–216, 2021.

- [33] NVIDIA. NVIDIA A100. <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-nvidia-us-2188504-web.pdf>, 2024.
- [34] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- [35] Google Compute Platform. Spot vms pricing. <https://cloud.google.com/spot-vms/pricing?hl=en>, 2025.
- [36] PyTorch. torch.distributed.elastic. <https://docs.pytorch.org/docs/stable/distributed.elastic.html>, 2024.
- [37] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [38] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE, 2020.
- [39] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 3505–3506, 2020.
- [40] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. Zero-offload: Democratizing billion-scale model training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 551–564, 2021.
- [41] Foteini Strati, Paul Elvinger, Tolga Kerimoglu, and Ana Klimovic. ML training with cloud GPU shortages: Is cross-region the answer? In *Proceedings of the 4th Workshop on Machine Learning and Systems*, pages 107–116, 2024.
- [42] Foteini Strati, Michal Friedman, and Ana Klimovic. Pccheck: Persistent concurrent checkpointing for ML. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pages 811–827, 2025.
- [43] Foteini Strati, Zhendong Zhang, George Manos, Ixeia Sánchez Pérez, Qinghao Hu, Tiancheng Chen, Berk Buzcu, Song Han, Pamela Delgado, and Ana Klimovic. Sailor: Automating distributed training over dynamic, heterogeneous, and geo-distributed clusters. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, pages 204–220, 2025.
- [44] John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali, and Guoqing Harry Xu. Bamboo: Making preemptible instances resilient for affordable training of large DNNs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 497–513, 2023.
- [45] Marcel Wagenländer, Guo Li, Bo Zhao, Luo Mai, and Peter Pietzuch. Tenplex: Dynamic parallelism for deep learning using parallelizable tensor collections. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 195–210, 2024.
- [46] Marcel Wagenländer, Luo Mai, Guo Li, and Peter Pietzuch. Spotnik: Designing distributed machine learning for transient cloud resources. In *12th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 20)*, 2020.
- [47] Guanhua Wang, Olatunji Ruwase, Bing Xie, and Yuxiong He. Fast-persist: Accelerating model checkpointing in deep learning. *arXiv preprint arXiv:2406.13768*, 2024.
- [48] Zhuang Wang, Zhen Jia, Shuai Zheng, Zhen Zhang, Xinwei Fu, TS Eugene Ng, and Yida Wang. Gemini: Fast failure recovery in distributed training with in-memory checkpoints. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 364–381, 2023.
- [49] BigScience Workshop, Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, et al. Bloom: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100*, 2022.
- [50] Zhanghao Wu, Wei-Lin Chiang, Ziming Mao, Zongheng Yang, Eric Friedman, Scott Shenker, and Ion Stoica. Can't be late: optimizing spot instance savings under deadlines. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 185–203, 2024.
- [51] Zongheng Yang, Zhanghao Wu, Michael Luo, Wei-Lin Chiang, Romil Bhardwaj, Woosuk Kwon, Siyuan Zhuang, Frank Sifei Luan, Gautam Mittal, Scott Shenker, et al. *SkyPilot*: An intercloud broker for sky computing. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 437–455, 2023.
- [52] John W Young. A first order approximation to the optimum checkpoint interval. *Communications of the ACM*, 17(9):530–531, 1974.
- [53] Ruiqi Zhong, Dhruva Ghosh, Dan Klein, and Jacob Steinhardt. Are larger pretrained language models uniformly better? comparing performance at the instance level. *arXiv preprint arXiv:2105.06020*, 2021.